

1. IDENTIFICACAO DO PROJETO

Projeto:	PP_2026_001_FORENSIA
Titulo:	ForensIA - Sistema Forense de Deteccao de Conteudo Gerado por IA
Versao doc:	1.1 (atualizado em 12/06/2026)
Periodo:	05/02/2026 - 05/11/2026
Equipe:	E1: Backend Java E2: Motor IA Python E3: Frontend/QA/Docs
Status:	EM ANDAMENTO - Fase 5 (Motor Texto concluido)

2. OBJETIVO

Sistema forense para deteccao automatizada de conteudo gerado por IA em textos, audios, imagens e videos. Classifica como HUMANO (<30%), HIBRIDO (30-69%) ou IA_GERADO (>=70%) com modelo HuggingFace Transformers ou heurísticas estatísticas como fallback.

3. ESCOPO

3.1 Incluido

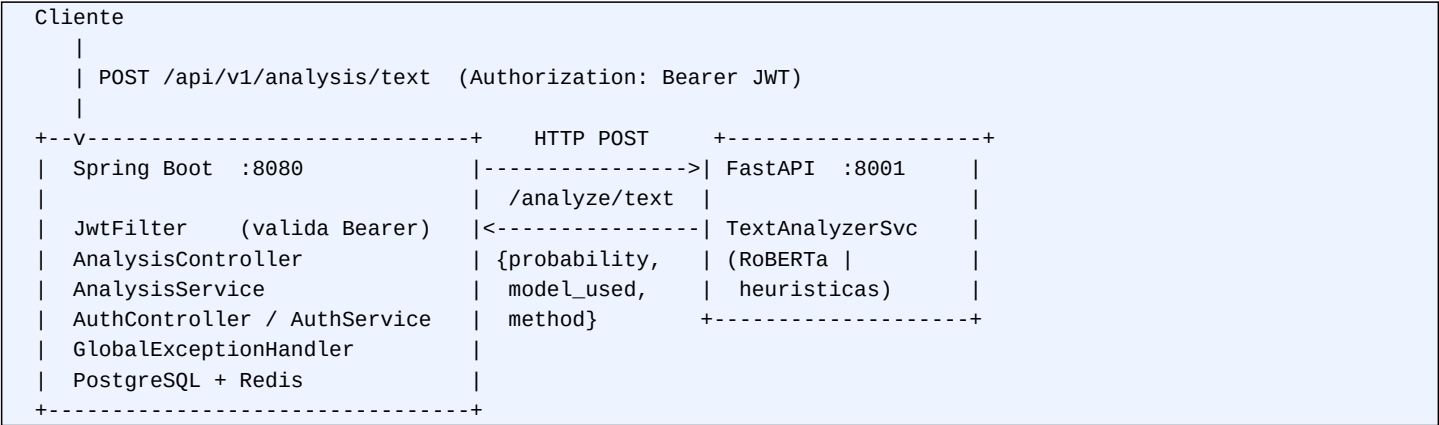
- [+] Cadastro e autenticacao de usuarios via JWT (Spring Security + BCrypt)
- [+] Analise de texto: modelo roberta-base-openai-detector + fallback heuristico
- [+] Classificacao HUMANO / HIBRIDO / IA_GERADO com warning acima de 70%
- [+] Persistencia de analises no PostgreSQL
- [+] API RESTful documentada (OpenAPI / Swagger em /docs)
- [+] Conteneurizacao Docker do motor Python
- [+] Tratamento centralizado de erros com respostas padronizadas (ApiResponse<T>)
- [Planejado] Analise de audio: PyTorch + librosa + MFCC
- [Planejado] Analise de imagem: PyTorch + torchvision ResNet-50
- [Planejado] Analise de video: OpenCV + PyTorch + analise de frames

3.2 Excluido

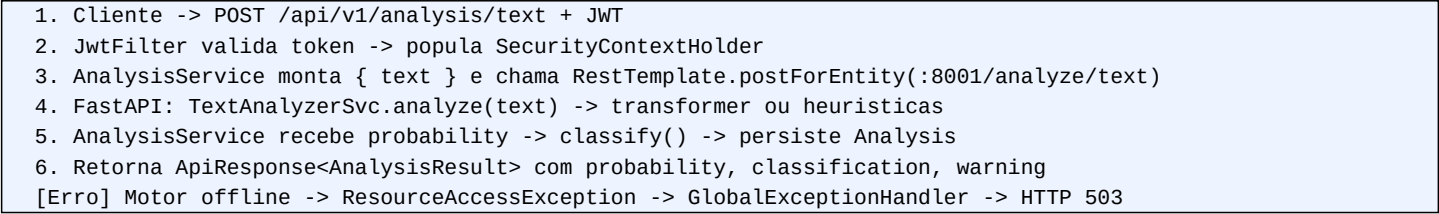
- [-] Interface Web/Desktop (definido na fase 5.5)
- [-] Suporte a idiomas alem de PT-BR e EN
- [-] Retraining do modelo ML (uso de modelo pre-treinado HuggingFace)

4. ARQUITETURA TECNICA

4.1 Visao Geral - Dois Microservicos



4.2 Fluxo de Analise (resumido)



4.3 Stack Tecnologica

Camada	Tecnologia	Versao	Resp.	Porta	Status
Backend API	Spring Boot	3.x	E1	8080	Parcial
Seguranca	Spring Security + JWT	-	E1	8080	Impl.
Banco	PostgreSQL 15	15	E1	5432	Config.
Cache	Redis	7	E1	6379	Config.
Motor IA	FastAPI	0.115	E2	8001	Pronto
Modelo ML	roberta-base-openai-detector	HuggingF	E2	-	Pronto
Framework ML	PyTorch + Transformers	2.6/4.46	E2	-	Pronto
Container	Docker	-	E1/E2	-	Motor OK
CI/CD	GitHub Actions	-	E1	-	Planej.

5. REQUISITOS FUNCIONAIS

ID	Nome	Prior.	Status
RF01	Cadastro de usuario	Alta	Implementado
Registra usuario com username/email/senha(BCrypt). Rejeita duplicatas. Valida forza da senha.			
RF02	Autenticacao via JWT	Alta	Implementado
Autentica username+senha e retorna token JWT Bearer. Token invalido gera HTTP 401.			
RF03	Analise de texto	Alta	Implementado
Recebe texto (min 5 chars), envia ao motor Python, retorna probability (0.0-1.0) e classification.			
RF04	Classificacao de conteudo	Alta	Implementado
HUMANO (<30%), HIBRIDO (30-69%), IA_GERADO (>=70%). Warning incluso quando probability >= 0.70.			
RF05	Persistencia de analises	Alta	Implementado
Resultado (username, probability, file_type, created_at) armazenado no PostgreSQL.			
RF06	Health check do motor IA	Alta	Implementado
GET /health retorna {status:ok}. Backend trata indisponibilidade do motor com HTTP 503.			
RF07	Fallback heuristico	Alta	Implementado
Quando modelo ML indisponivel, motor usa 5 heurísticas ponderadas (burstiness, TTR, etc.).			
RF08	Historico de analises	Media	Planejado
Usuario autenticado consulta suas analises paginadas. Endpoint GET /api/v1/analysis/history.			
RF09	Analise de audio	Alta	Planejado
Aceita arquivo de audio; extrai MFCC via librosa; classifica com modelo PyTorch.			
RF10	Analise de imagem	Alta	Planejado
Aceita imagem JPG/PNG; processa com ResNet-50 via torchvision; retorna probability.			
RF11	Analise de video	Alta	Planejado
Extrai frames com OpenCV; analisa imagem + audio separado; combina probabilidades.			
RF12	Interface do usuario	Alta	Planejado
Frontend permite upload de conteudo e exibe resultado com grafico e historico.			

6. REQUISITOS NAO-FUNCIONAIS

ID	Categoria	Descricao	Metrica / Criterio
RNF01	Desempenho	Resposta da analise de texto em tempo aceitavel	< 3s para textos ate 512 tokens
RNF02	Disponibilidade	Sistema operacional na maior parte do tempo	Uptime >= 99.5% em producao
RNF03	Seguranca	Comunicacao criptografada; autenticacao robusta	HTTPS/TLS 1.3; JWT HS256; BCrypt >..
RNF04	Conformidade	Adequacao a LGPD e GDPR para dados pessoais	DPIA realizado; texto nao persisti..
RNF05	Escalabilidade	Suporta aumento de carga sem refatoracao	Docker Compose; horizontal scaling
RNF06	Portabilidade	Executavel em qualquer ambiente com Docker	docker-compose up sobe todo o stack
RNF07	Manutencao	Codigo legivel, testado e documentado	Cobertura >= 70% (JUnit5 + pytest)
RNF08	Precisao ML	Modelo com acuracia aceitavel em producao	F1-score >= 0.80 no conjunto de te..
RNF09	Tolerancia	Falha do motor Python nao derruba o backend	GlobalExceptionHandler -> HTTP 503
RNF10	Auditoria	Todas as analises rastreadas por usuario	username + timestamp imutaveis na ..

7. REGRAS DE SISTEMA

ID	Regra de Negocio	Componente	Tipo
RS01	Senha deve conter maiuscula, minuscula e digito	RegisterRequest	Validacao
RS02	Username e Email devem ser unicos no banco	AuthService / UserRepository	Integridade
RS03	JWT obrigatorio em /api/v1/**	JwtFilter / SecurityConfig	Autenticacao
RS04	Texto para analise deve ter >= 5 caracteres	FastAPI TextRequest validator	Validacao
RS05	probability >= 0.70 gera campo warning na resposta	AnalysisService.analyzeText()	Negocio
RS06	Motor Python indisponivel retorna HTTP 503	GlobalExceptionHandler	Resiliencia
RS07	Modelo transformer trunca texto em 512 tokens	TextAnalyzerService._load_mod..	ML
RS08	Fallback ativado se modelo ML falhar ao carregar	TextAnalyzerService._load_mod..	ML
RS09	Role padrao de novo usuario e USER	AuthService.register()	Negocio
RS10	Toda analise persiste username+probability+file_..	AnalysisService + AnalysisRep..	Auditoria

8. MODELO DE DADOS

8.1 Diagrama ER (descritivo)

+-----+ users +-----+	+-----+ analyses +-----+
PK id BIGSERIAL	PK id BIGSERIAL
username ----+--1-----N->	username VARCHAR
email_user	probability FLOAT
password_user	file_type VARCHAR
role	role VARCHAR
created_at	created_at TS
+-----+	+-----+
Referencia por username (sem FK formal para flexibilidade)	

8.2 Schema - Tabela users

Coluna	Tipo	Restricoes	Descricao
id	BIGSERIAL	PRIMARY KEY	Identificador unico autoincrement
username	VARCHAR(50)	NOT NULL UNIQUE	Nome de usuario - unico no sist..
email_user	VARCHAR(100)	NOT NULL UNIQUE	Email - unico no sistema
password_user	VARCHAR(255)	NOT NULL	Senha hasheada com BCrypt
role	VARCHAR(20)	NOT NULL DEFAULT 'USER'	Perfil: USER ou ADMIN
created_at	TIMESTAMP	DEFAULT NOW()	Data/hora de criacao

8.3 Schema - Tabela analyses

Coluna	Tipo	Restricoes	Descricao
id	BIGSERIAL	PRIMARY KEY	Identificador unico autoincrement
username	VARCHAR(50)	NOT NULL	Usuario que realizou a analise
probability	DOUBLE PRECISION	NOT NULL	Probabilidade: 0.0 (humano) a 1..
file_type	VARCHAR(20)	DEFAULT 'TEXT'	TEXT AUDIO IMAGE VIDEO
role	VARCHAR(20)	-	Role do usuario no momento
created_at	TIMESTAMP	DEFAULT NOW()	Timestamp da analise

8.4 Entidades JPA (Java)

```

@Entity @Table(name='users')
class User {
    @Id @GeneratedValue Long id;
    String username;        // unique=true no @Column
    String email_user;
    String password_user;   // BCrypt encoded
    String role;            // @Enumerated REMOVIDO - String simples
}

@Entity @Table(name='analyses')
class Analysis {
    @Id @GeneratedValue Long id;
    String username;        // referencia ao user (sem FK)
    Double probability;
    String file_Type;       // 'TEXT' | 'AUDIO' | 'IMAGE' | 'VIDEO'
    String role;
    // created_at via @PrePersist ou DEFAULT NOW()
}

```

9. CASOS DE USO

Atores: Usuario (qualquer pessoa), Sistema (automatico), Motor IA (FastAPI)

UC01 - Registrar Usuario

Ator:	Usuario (nao autenticado)
Pre-cond.:	Sistema operacional; usuario sem conta
Fluxo Principal:	
1. Usuario envia POST /auth/register com { Username, Email_user, Password_user }	
2. AuthController valida campos (Bean Validation)	
3. AuthService verifica username unico (existsByUsername)	
4. AuthService verifica email unico (existsByEmail_user)	
5. AuthService codifica senha com BCryptPasswordEncoder	
6. AuthService salva User com role='USER'	
7. Retorna HTTP 200 { sucess:true, data:{ message:'Usuario registrado com sucesso' } }	
Fluxo Alternativo / Excecao:	
2a. Campo invalido (senha fraca) -> 400 Bad Request com detalhes do erro	
3a. Username duplicado -> 200 { sucess:false, 'Nome de usuario ja esta em uso' }	
4a. Email duplicado -> 200 { sucess:false, 'Email ja esta em uso' }	
Pos-cond.:	Novo usuario criado e disponivel para autenticacao

UC02 - Autenticar Usuario (Login)

Ator:	Usuario (cadastrado)
Pre-cond.:	Usuario possui conta ativa
Fluxo Principal:	
1. Usuario envia POST /auth/login com { Username, Password_user }	
2. AuthController valida campos	
3. AuthService chama AuthenticationManager.authenticate(UsernamePasswordAuthToken)	
4. Spring Security carrega UserDetails via UserDetailsServiceImpl	
5. BCryptPasswordEncoder compara senha com hash no banco	
6. JwtUtil.generateToken(username) gera JWT assinado HS256	
7. Retorna HTTP 200 { token:'eyJ...', type:'Bearer', username:'...' }	
Fluxo Alternativo / Excecao:	
5a. Senha incorreta -> BadCredentialsException -> 200 { sucess:false, 'Credenciais invalidas' }	
2a. Campo ausente -> 400 Bad Request	
Pos-cond.:	JWT valido emitido; usuario acessa endpoints protegidos

UC03 - Analisar Texto

Ator:	Usuario (autenticado com JWT)
Pre-cond.:	JWT valido obtido via UC02
Fluxo Principal:	
1. POST /api/v1/analysis/text com Authorization: Bearer <token> e { Text:'...' }	
2. JwtFilter extrai e valida token; popula SecurityContextHolder	
3. AnalysisController recebe TextRequest	
4. AnalysisService.analyzeText(username, request)	
5. HTTP POST a http://localhost:8001/analyze/text com { text:'...' }	
6. Motor retorna { probability, model_used, method }	
7. classify(): <0.30=HUMANO 0.30-0.69=HIBRIDO >=0.70=IA_GERADO	
8. Analysis persistida no PostgreSQL	
9. Retorna HTTP 200 { probability, classification, warning?, created_at }	
Fluxo Alternativo / Excecao:	
2a. JWT invalido/expirado -> JwtFilter -> 401 Unauthorized	
5a. Motor offline -> ResourceAccessException -> GlobalExceptionHandler -> 503	
3a. Texto < 5 chars -> FastAPI validator -> 422 Unprocessable Entity	

6a. Modelo ML indisponivel -> motor usa fallback heuristico (method='statistical')	
Pos-cond.:	Analise registrada; usuario recebe classificacao e probabilidade
UC04 - Verificar Saude do Motor IA	
Ator:	Sistema (backend Java) ou Administrador
Pre-cond.:	Motor Python iniciado
Fluxo Principal:	
1. GET http://localhost:8001/health	
2. FastAPI retorna HTTP 200 { status:'ok', service:'ForensIA Engine' }	
Fluxo Alternativo / Excecao:	
2a. Motor offline -> Connection refused -> tratado como 503 em UC03	
Pos-cond.:	Disponibilidade do motor confirmada

10. DIAGRAMAS DE SEQUENCIA (ASCII)

10.1 Registro de Usuario (UC01)

Cliente	AuthController	AuthService	UserRepository	BD
--POST /register-->				
	--register(req)-->			
		--existsByUsername()----->		
		<--false-----		
		--existsByEmail()----->		
		<--false-----		
		--BCrypt.encode(password)		
		--save(user)----->		
			--INSERT-->	
	<--ApiResponse ok-			
<--200 {sucess:true}				

10.2 Login e JWT (UC02)

Cliente	AuthController	AuthService	AuthManager	JwtUtil	BD
--POST /login-->					
	--login(req)--->				
		--authenticate(token)--->			
			--loadUser()----->		
			<--User-----		
			--BCrypt.matches()		
		<--Authentication ok---			
		--generateToken(user)-->			
		<--JWT-----			
	<--ApiResponse--				
<--200 {token}--					

10.3 Analise de Texto (UC03)

Cliente	JwtFilter	AnalysisCtrl	AnalysisSvc	FastAPI	AnalysisRepo
--POST text+JWT----->					
	--validate JWT				
	--set SecurityContext				
	----->				
		--analyzeText(user, req)--->			
			--POST /analyze/text----->		
			<--{probability:0.87}-----		
			--classify()->IA_GERADO		
			--save(Analysis)----->		
		<--ApiResponse<AnalysisResult>-----			
<--200 {probability, classification, warning}-----					

10.4 Motor Python Indisponivel (Cenario de Erro)

Cliente	JwtFilter	AnalysisCtrl	AnalysisSvc	FastAPI	GlobalExcHandler
--POST text+JWT----->					
	--validate--				
	----->	--analyzeText()-->			
			--POST :8001 X (offline)		
			--ResourceAccessException-->		
		<----handleAiEngineUnavailable()-----			
<--HTTP 503 { sucess:false, Motor de IA indisponivel }-----					

11. COMPONENTES IMPLEMENTADOS (Jun/2026)

11.1 Motor Python - CONCLUIDO (40/40h)

Microservico FastAPI em python-engine/. TextAnalyzerService singleton carregado na inicializacao.

Modo Transformer: pipeline HuggingFace roberta-base-openai-detector, max_length=512.

Modo Heuristico: 5 features ponderadas calculadas sobre o texto.

Feature	Peso	Indicador de IA	Formula / Limiar
low_burstiness	30%	Variacao baixa no tamanho das frases	variance/mean < 8.0
many_transitions	25%	Conectivos formais (furthermore, portanto...)	min(1.0, count/3.0)
high_ttr	20%	Type-Token Ratio alto (vocabulario diverso)	len(unique)/len(words) > 0.65
long_sentences	15%	Frases longas e completas	avg palavras/frase > 18
no_personal_pronouns	10%	Escrita impessoal (sem eu/nos/my/our)	1.0 se nenhum pronome

11.2 Backend Java - Auth + Analysis (28/40h)

Arquivo	Pacote	Responsabilidade
UserDetailsServiceImpl	service	Implementa UserDetailsService; carrega User para S..
AuthService	service	Registro (BCrypt) + login (JWT); verifica unicidade
AnalysisService	service	Chama motor Python via RestTemplate; classifica; p..
AuthController	controller	POST /auth/register e POST /auth/login
AnalysisController	controller	POST /api/v1/analysis/text (requer JWT)
GlobalExceptionHandler	exception	Trata erros: 400 / 401 / 403 / 503 / 500

11.3 Correcoes Aplicadas - java-fixes/ (9 bugs)

Arquivo	Problema Original	Correcao Aplicada
SecurityConfig.java	Path /auth/** (typo na rota)	/auth/**
JwtFilter.java	username == null (logica invertida)	username != null
RegisterRequest.java	Regex [?=.*\d] (classe char errada)	(?=.*\d) lookahead
LoginsRequest.java	Mesmo regex invalido [?=.*\d]	(?=.*\d) lookahead
Analysis.java	unique=true em campo username	Removido
Analysis.java	@Enumerated em campo String	Removido
User.java	@Enumerated em campo String role	Removido
User.java	import javax.swing.* (errado)	Removido
RestTemplate.java	Classe vazia conflita com bean Spring	Deletado; adicionado RestTempl..

12. STATUS DAS ATIVIDADES - CRONOGRAMA

Atualizacao: 12/06/2026 | Motor Texto: CONCLUIDO | Motor Audio: ATRASADO (1 dia)

Seq	Atividade	Est.	Real.	Inicio	Termino	Resp.	Status
1	Kick-off do Projeto	8	8	05/02	12/02	E1,2,3	FEITO
2	Identificacao de Stakeholders	8	8	10/02	17/02	E1	FEITO
3	Definicao de Escopo	16	16	19/02	05/03	E1	FEITO
4	Plano de Riscos	8	8	26/02	05/03	E1	FEITO
5	Plano de Comunicacao	4	4	26/02	03/03	E1	FEITO
6	Elicitacao de Requisitos	24	24	05/03	19/03	E1,E3	FEITO
7	Documentacao SRS - Marco 1	24	24	12/03	26/03	E3	FEITO
8	Regras de Sistema	16	16	19/03	29/03	E3	FEITO
9	Modelagem de Dados	16	7	02/04	12/04	E1	PARCIAL
10	Design da API REST	16	5	09/04	19/04	E1	PARCIAL
11	Prototipo Interface - Marco 2	16	2	09/04	23/04	E3	PARCIAL
12	Motor Deteccao Texto	40	40	30/04	12/06	E2	FEITO
13	Motor Deteccao Audio	40	-	21/05	11/06	E2	ATRASADO
14	Motor Deteccao Imagem	40	-	11/06	02/07	E2	PENDENTE
15	Motor Deteccao Video	40	-	02/07	23/07	E2	PENDENTE
16	API RESTful Spring Boot	40	28	21/05	02/07	E1	PARCIAL
17	Interface Usuario - Marco 3	40	-	02/07	30/07	E3	PENDENTE
18	Testes Unitarios	24	-	20/08	03/09	E1,E2	PENDENTE
19	Testes de Integracao	20	-	27/08	06/09	E1,E3	PENDENTE
20	Testes de Sistema	20	-	03/09	13/09	E3	PENDENTE
21	Testes do Modelo ML	16	-	03/09	13/09	E2	PENDENTE
22	Testes de Desempenho	12	-	10/09	17/09	E1,E3	PENDENTE
23	Testes de Seguranca - Marco 4	12	-	10/09	17/09	E3	PENDENTE
24	Configuracao de Ambiente	16	-	24/09	04/10	E1	PENDENTE
25	Deploy Producao - Marco 5	16	-	01/10	08/10	E1	PENDENTE
26	Treinamento de Usuarios	8	-	08/10	13/10	E1,E3	PENDENTE
27	Manual do Usuario	16	-	15/10	29/10	E3	PENDENTE
28	Documentacao Tecnica Final	16	-	22/10	01/11	E1,2,3	PENDENTE
29	Licoes Aprendidas - Marco 6	8	-	01/11	06/11	E1,2,3	PENDENTE
TOTAL			580	190	33% concluido (190/580h)		

13. REFERENCIA DE API

13.1 Java Backend - http://localhost:8080

Metodo	Endpoint	Auth	Body / Descricao
POST	/auth/register	Nenhuma	{ Username, Email_user, Password_user } - Registra usuario
POST	/auth/login	Nenhuma	{ Username, Password_user } - Retorna JWT Bearer
POST	/api/v1/analysis/text	JWT	{ Text } - Analisa texto; retorna probability + classification

13.2 Motor Python - http://localhost:8001

Metodo	Endpoint	Auth	Body / Descricao
POST	/analyze/text	Nenhuma	{ text } - Analisa texto; retorna probability, model_used, me..
GET	/health	Nenhuma	Retorna { status:'ok', service:'ForensIA Engine' }

13.3 Exemplo de Requisicao - Analise de Texto

```
// POST /auth/register
{ "Username": "usuario01", "Email_user": "user@email.com", "Password_user": "Senha123" }

// POST /auth/login
{ "Username": "usuario01", "Password_user": "Senha123" }

// POST /api/v1/analysis/text (Header: Authorization: Bearer eyJ...)
{ "Text": "Furthermore, it is worth noting that the implementation requires..." }

// POST http://localhost:8001/analyze/text (sem JWT)
{ "text": "Furthermore, it is worth noting that the implementation requires..." }
```

13.4 Exemplo de Resposta

```
// 200 OK - Analise de texto (Java)
{ "sucess": true, "data": { "probability": 0.87, "classification": "IA_GERADO",
  "file_Type": "TEXT", "warning": "Alta probabilidade de conteudo gerado por IA",
  "created_at": "2026-06-12T10:30:00" }}

// 200 OK - Motor Python modo transformer
{ "probability": 0.87, "model_used": "openai-community/roberta-base-openai-detector",
  "method": "transformer", "label": "Fake", "confidence": 0.87 }

// 200 OK - Motor Python modo heuristico (fallback)
{ "probability": 0.75, "model_used": "statistical-heuristics", "method": "statistical",
  "features": { "low_burstiness":1.0, "many_transitions":0.67, "high_ttr":1.0,
    "long_sentences":0.0, "no_personal_pronouns":1.0 }}

// 503 - Motor offline
{ "sucess": false, "data": "Motor de IA indisponivel: Connection refused" }
```

13.5 Classificacao por Probabilidade

Faixa	Classificacao	Descricao
probability < 0.30	HUMANO	Alta confianca de escrita humana
0.30 <= probability < 0.70	HIBRIDO	Conteudo misto ou inconclusivo
probability >= 0.70	IA_GERADO	Alta probabilidade de IA; campo warning incluso

14. REGISTRO DE RISCOS

I = Impacto (1-5) | P = Probabilidade (1-5) | IR = I*P | IR>=12=ALTO, >=6=MEDIO

ID	Descricao	Cat.	I	P	IR	Plano de Resposta / Status
R01	Baixa acuracia do modelo em pro..	Tecnico	4	3	12	Mitigado: transformer + fallback heuristico
R02	Mudancas nos LLMs invalidam o m..	Externo	4	4	16	Aberto: retraining periodico planejado
R03	Nao conformidade com LGPD/GDPR	Legal	5	2	10	Aberto: DPIA pendente; texto nao persiste
R04	Atraso na infraestrutura de nuvem	Org.	3	3	9	Mitigado: Docker Compose local disponivel
R05	Saida de membro-chave da equipe	RH	4	2	8	Mitigado: java-fixes/ e java-src/ entregues
R06	Escopo nao aprovado pelo patroc..	Gestao	3	2	6	Aberto: gates por fase; reunioes semanais
R07	Motor Python offline degrada a ..	Tecnico	4	3	12	Mitigado: GlobalExceptionHandler -> 503
R08	Vazamento de dados submetidos	Seguranca	5	2	10	Aberto: HTTPS/TLS 1.3 planejado

15. ANALISE PERT - DURACOES POR FASE CRITICA

Formula: PERT = (O + 4M + P) / 6 | O=Otimista M=Mais Provavel P=Pessimista

Fase / Atividade	O	M	P	PERT	Resp.	Status
Motor de Deteccao - Texto	4	8	12	8.0	E2	FEITO
Motor de Deteccao - Audio	4	8	12	8.0	E2	ATRASADO
Motor de Deteccao - Imagem	4	8	14	8.3	E2	PENDENTE
Motor de Deteccao - Video	4	8	14	8.3	E2	PENDENTE
API RESTful Spring Boot	4	6	10	6.3	E1	PARCIAL
Interface do Usuario	2	5	8	5.0	E3	PENDENTE
Testes e QA	1	2	4	2.2	E1/E3	PENDENTE
Implantacao e Deploy	1	2	3	2.0	E1	PENDENTE
DURACAO TOTAL ESTIMADA (PERT)				48.1 sem.		

16. LEGENDA E CONVENCOES

E1 = Estudante 1 - Lider / Backend Java (Spring Boot + PostgreSQL + Redis + JWT)

E2 = Estudante 2 - Motor IA (FastAPI + HuggingFace + PyTorch + librosa + OpenCV)

E3 = Estudante 3 - Interface / Testes / Documentacao

STACK: Backend -> Java 21 + Spring Boot 3.x + PostgreSQL 15 + Redis 7

Motor IA -> Python 3.11 + FastAPI 0.115 + Transformers 4.46 + PyTorch 2.6

Audio (planejado) -> librosa + MFCC feature extraction

Imagem (planejado) -> torchvision + ResNet-50

Video (planejado) -> OpenCV + frame sampling

CI/CD -> Docker + GitHub Actions

17. HISTORICO DE VERSOES DO DOCUMENTO

Versao	Data	Autor	Descricao
1.0	05/02/2026	E1,E2,E3	Versao inicial - Identificacao, cronograma, riscos, PERT
1.1	12/06/2026	E1,E2	Motor Texto concluido; 9 bugs Java corrigidos; Auth+Analysis impleme..

Documento gerado em 12/06/2026 18:34 - ForensIA v1.1